Tu responsabilidad y el recorrido completo

Archivo: src/pipeline.py | Función o clase: _run | Inicio: línea 22

Tu parte mide la calidad, identifica duplicados y trata faltantes y extremos. Avilix conecta esas funciones con lectura, integración y transformación. Leo define validadores y persistencia. Los roles son responsabilidades del equipo; no significan autoría exclusiva de cada línea.

Lee en este orden: src/calidad.py, src/duplicados.py y src/limpieza.py. Después abre _run en src/pipeline.py para ver dónde se invocan. El proceso es lectura, perfilado, deduplicación, validación, catálogo, imputación, IQR, escalamiento y carga.

Un registro es una fila u objeto. Un lote es un archivo procesado. candidatos contiene filas que pueden continuar; rechazados conserva filas que deben ir a cuarentena; parametros guarda cómo se calcularon los cambios.

Fragmento real, líneas 22-31. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
22 def _run(rows,source,db,metadata,identity):
23     version,catalog=obtener_catalogo(db)
24     metadata.update(catalog_version=version,rules_version=RULES,process_version=PROCESS)
25     before=perfilar([r if isinstance(r,dict) else {'_valor':r} for r in rows],source)
26     # Duplicación exacta antes de incorporar posiciones, reutilizando el módulo de Maki.
27     dedup=separar_exactos(rows); omitted_positions={r['posicion'] for r in
   dedup['omitidos']}
28     candidates=[];omitted=[];bad=[]
29     for i,raw in enumerate(rows,1):
30         r={**(raw if isinstance(raw,dict) else
   {}),'original_json':raw,'fila_origen':i+metadata.get('offset',1)-1,
31
   'archivo_origen':metadata['archivo'],'origen_ingreso':metadata.get('origen','ARCHIVO')}
```

Perfilado: medir antes de cambiar datos

Archivo: src/calidad.py | Función o clase: perfilar | Inicio: línea 44

Entrada: una lista de diccionarios crudos y el nombre de la fuente. Salida: filas, campos, duplicados_exactos y metricas_por_campo. La función no modifica los registros. _tipo_observado identifica tipos Python; _es_vacio distingue espacios de valores presentes.

El bucle recorre cada campo. Un None suma nulos; una cadena vacía o con espacios suma vacios. Counter acumula tipos y categorías. pct_nulos divide nulos entre todas las filas de esa fuente y multiplica por 100; una lista vacía devuelve 0 para evitar división entre cero.

En los originales hay 27 montos ATM nulos de 620 filas: 4.35 %. No son 27 filas necesariamente imputadas: algunas pueden ser duplicadas o rechazadas antes. El perfil queda en evidencia_lotes.datos, dentro de perfil_inicial.

Fragmento real, líneas 62-77. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
62              else:
63                  tipos[_tipo_observado(v)] += 1
64                  valores_no_nulos.append(v)
65          metricas_por_campo[campo] = {
66              "nulos": nulos,
67              "vacios": vacios,
68              "no_nulos": total - nulos - vacios,
69              "pct_nulos": round(nulos / total * 100, 2) if total else 0.0,
70              "pct_vacios": round(vacios / total * 100, 2) if total else 0.0,
71              "tipos_observados": dict(tipos),
72              "categorias_top": _top_categorias(valores_no_nulos),
73          }
74
75      serializados = [json.dumps(r, sort_keys=True, default=str) for r in registros]
76      conteo = Counter(serializados)
77      duplicados_exactos = sum(c - 1 for c in conteo.values() if c > 1)
```

Duplicados: conservar una copia y registrar el resto

Archivo: src/duplicados.py | Función o clase: separar_exactos | Inicio: línea 13

_clave_exacta usa json.dumps con sort_keys=True: el orden de las claves no cambia la comparación. vistos recuerda la primera posición. Si la clave ya apareció, se registra posicion, referencia_conservada, motivo y original_json en omitidos.

Salida: unicos y omitidos. En _run se usan las posiciones para construir la bitácora antes de añadir metadatos que harían distintas dos copias. Los 620 registros ATM contienen 20 copias adicionales; se conservan 600 candidatos antes de otras validaciones.

detectar_conflictos también existe en este archivo como utilidad probada. El pipeline integrado resuelve conflictos en _run usando huella_origen: mismo ID con contenido distinto va a cuarentena. No deduplicamos solo por monto o cuenta.

Fragmento real, líneas 13-30. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
13 def separar_exactos(registros: list[dict]) -> dict:
14     """Conserva la primera aparición exacta; el resto va a omitidos."""
15     vistos: dict[str, int] = {}
16     unicos: list[dict] = []
17     omitidos: list[dict] = []
18 
19     for pos, r in enumerate(registros, start=1):
20         clave = _clave_exacta(r)
21         if clave in vistos:
22             omitidos.append({
23                 "posicion": pos,
24                 "referencia_conservada": vistos[clave],
25                 "motivo": "DUPLICADO_EXACTO",
26                 "original_json": r,
27             })
28         else:
29             vistos[clave] = pos
30             unicos.append(r)
```

Imputación: decidir cuándo usar la mediana

Archivo: src/limpieza.py | Función o clase: imputar | Inicio: línea 17

_key construye los grupos: monto ATM por operación y estado; monto digital por moneda original y estado; comisión ATM por operación y cajero conocido. Los importes digitales ya están en MXN, pero se conserva el grupo de moneda original.

groups reúne valores observados válidos. median(values) calcula el centro; también se guardan media y tamano_grupo. Se exigen cinco observaciones. Un monto corrupto, negativo o infinito no se convierte en una ausencia imputable.

MCAR significa ausencia independiente de las variables; MAR permite dependencia de variables observadas. El agrupamiento usa una hipótesis MAR operativa, no prueba su causa. La mediana se prefiere por su menor sensibilidad a extremos. No se reconstruyen importes con cantidad por precio porque las fuentes no tienen esos campos.

Fragmento real, líneas 17-31. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
17 def imputar(registros,parametros=None):
18     params={} if parametros is None else parametros
19     if parametros is None:
20         groups={}
21         for r in registros:
22             for field in ('monto_mxn','comision_aplicada'):
23                 if field=='comision_aplicada' and (r.get('fuente')!='ATM' or not
   r.get('id_cajero') or r.get('id_cajero')=='DESCONOCIDO'): continue
24                 if r.get('tipo_operacion')=='CONSULTA_SALDO' and field=='monto_mxn':
   continue
25                 value=r.get(field)
26                 if _es_numero_valido(value,field=='comision_aplicada') and not
   r.get('_cambios_imputacion'):
27                     groups.setdefault(_key(r,field),[]).append(Decimal(str(value)))
28         for key,values in groups.items():
29             params[key]=dict(mediana=median(values),tamano_grupo=len(values),media=sum(value
   s)/len(values))
30     candidates,rejected=[],[]
31     for raw in registros:
```

Imputación: aplicar y dejar evidencia

Archivo: src/limpieza.py | Función o clase: imputar | Inicio: línea 17

Para cada fila se crea una copia. Si el monto de CONSULTA_SALDO falta, se asigna cero. En otras operaciones se busca la mediana del grupo; si hay menos de cinco observaciones, se registra SIN_BASE_PARA_IMPUTAR. Las comisiones pueden ser cero.

changes conserva campo, antes, despues, regla y grupo. El resultado devuelve candidatos, rechazados y parametros. Algunas etiquetas como DESCONOCIDO y SIN_CONCEPTO se aplican antes en los validadores; el pipeline conserva esas normalizaciones por separado.

Ejemplo oficial ATM-TX-00034: monto original null, operación PAGO_SERVICIOS, estado FALLO_FONDOS. El grupo tiene 19 observaciones, mediana 4000 y media aproximada 5894.74. Se asignan 4000 MXN sin borrar el null original.

Fragmento real, líneas 42-55. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
42                     r[field]=Decimal(0);rule='CONSULTA_SALDO_CERO'
43                 else:
44                     info=params.get(_key(r,field),{})
45                     if info.get('tamano_grupo',0)<5:
46                         errors.append(field);continue
47                     r[field]=Decimal(str(info['mediana']));rule='MEDIANA_GRUPO'
48                 changes.append(dict(campo=field,antes=value,despues=r[field],regla=rule,grup
   o=_key(r,field)))
49             elif not _es_numero_valido(value,field=='comision_aplicada' or
   r.get('tipo_operacion')=='CONSULTA_SALDO'):
50                 rejected.append(dict(original_json=raw,motivo='FORMATO_NUMERICO_INVALIDO',ca
   mpo=field));break
51         else:
52             if errors:
53                 rejected.append(dict(original_json=raw,motivo='SIN_BASE_PARA_IMPUTAR',campo=
   ','.join(errors)));continue
54             r['_cambios_imputacion']=changes;candidates.append(r)
55     return dict(candidatos=candidates,rechazados=rejected,parametros=params)
```

IQR: limitar la variable analítica

Archivo: src/limpieza.py | Función o clase: aplicar_iqr | Inicio: línea 57

Se agrupa por fuente y operación. percentil, en src/transformacion.py, ordena valores e interpola Q1 y Q3. IQR = Q3 - Q1; límites = Q1 - 1.5 IQR y Q3 + 1.5 IQR, con límite inferior al menos cero.

El ajuste excluye consultas de saldo y montos imputados con mediana. Con menos de cinco observaciones o IQR cero se conserva el monto sin recorte y se guarda el motivo. En la segunda mitad de aplicar_iqr, min(hi, max(lo, value)) limita el valor.

Solo cambia monto_analitico_mxn; monto_mxn conserva el importe monetario. SPEI-2026-00003 conserva 552091.22 MXN y obtiene 69304.92 MXN analíticos. es_outlier queda true. Un extremo válido no se elimina ni se considera fraude automáticamente.

Fragmento real, líneas 57-72. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
57 def aplicar_iqr(registros,parametros=None):
58     params={} if parametros is None else parametros
59     if parametros is None:
60         groups={}
61         for r in registros:
62             key=r.get('fuente','')+'|'+r.get('tipo_operacion','');groups.setdefault(key,[])
63             imputed=any(c['campo']=='monto_mxn' and c['regla']=='MEDIANA_GRUPO' for c in
   r.get('_cambios_imputacion',[]))
64             if r.get('tipo_operacion')!='CONSULTA_SALDO' and not imputed and
   _es_numero_valido(r.get('monto_mxn')): groups[key].append(r['monto_mxn'])
65         for key,values in groups.items():
66             if len(values)<5:
67
   params[key]=dict(recorte=False,motivo='GRUPO_MENOR_A_5',n=len(values));continue
68             a,b=percentil(values,'.25'),percentil(values,'.75');iqr=b-a
69             params[key]=dict(recorte=iqr>0,motivo='IQR' if iqr else 'IQR_CERO',n=len(values)
   ,q1=a,q3=b,iqr=iqr,limite_inferior=max(Decimal(0),a-
   Decimal('1.5')*iqr),limite_superior=b+Decimal('1.5')*iqr)
70     candidates=[]
71     for raw in registros:
72         r=dict(raw);value=Decimal(str(r['monto_mxn']));info=params.get(r.get('fuente','')+'|
   '+r.get('tipo_operacion',''),{})
```

Comparación, comprobaciones y defensa

Archivo: src/calidad.py | Función o clase: comparar_calidad | Inicio: línea 96

comparar_calidad devuelve antes, despues y efecto_por_etapa. La población inicial incluye todas las filas; la final contiene aceptadas. Por eso una caída de nulos no se atribuye solo a imputación: también intervienen rechazos y duplicados.

Pruebas necesarias: tests/test_calidad.py comprueba que el perfilado no altera la entrada y que se conservan duplicados; tests/test_limpieza.py comprueba mediana, grupo insuficiente, consulta cero y capping sin pérdida de filas. Se conservan porque respaldan la rúbrica.

Para practicar: .venv/Scripts/python.exe docs/guias/consultar_ejemplos.py maki. Busca ejemplo_imputacion y ejemplo_iqr. Explica la diferencia entre observado e imputado y reproduce el límite IQR. No memorices cifras de archivos nuevos: estas corresponden a los originales.

Fragmento real, líneas 96-109. Las líneas largas se continúan sin número; consulta el archivo para ver la función completa.

```
 96 def comparar_calidad(antes: dict, despues: dict, resumen: dict) -> dict:
 97     """Compara perfilados antes/después. Genera métricas para la fase de reporte (M4)."""
 98     return {
 99         "antes": {
100             "filas": antes["filas"],
101             "duplicados_exactos": antes["duplicados_exactos"],
102             "campos_con_nulos": {
103                 c: m["nulos"] for c, m in antes["metricas_por_campo"].items() if m["nulos"]
    > 0
104             },
105         },
106         "despues": {
107             "filas": despues["filas"],
108             "duplicados_exactos": despues["duplicados_exactos"],
109             "campos_con_nulos": {
```

Repaso y archivos para entregar

ETL significa extraer, transformar y cargar. Puedes explicar tu módulo indicando su entrada, la decisión que toma, su salida y la evidencia que conserva. No basta con leer nombres de técnicas.

Los ejemplos de estas guías corresponden a datasets/catalogo_cuentas.csv, datasets/transacciones_atm.xlsx y datasets/transferencias_digitales.json. Con archivos nuevos cambian las estadísticas y resultados. La tasa académica USD/MXN es fija: 19.50.

Entrega al profesor: enlace al repositorio Git, entrega/pipeline_banco.sqlite y entrega/REPORTE_FINAL.pdf. Dentro del repositorio quedan reglas, diagrama, código y pruebas de resiliencia. Las guías de docs/guias son material de estudio del equipo.

El video y la presentación indicados posteriormente por el profesor se preparan aparte; esta guía no equivale a haber grabado o expuesto. docs/GUION_VIDEO.md propone el recorrido. La interfaz es un apoyo visual para demostrar el pipeline.

Para localizar el código, abre el archivo .py en el editor y busca el nombre de función indicado. Las líneas corresponden a esta versión y pueden moverse con futuros cambios. Los fragmentos se extraen del código al regenerar las guías; no son pseudocódigo.

Referencias: guía del profesor, puntos 4 y 6; REGLAS_DE_NEGOCIO.md; docs/CONTRATO_DATOS.md; código y pruebas citados; docs/evidencia/resultados.json; base SQLite de entrega. Regla vigente 1.5, proceso 2.1.